

SOLAR BATTERY CHARGER USING BUCK CONVERTER AND MPPT ALGORITHM

SWITCHED MODE POWER CONVERSION (EEE4014) PROJECT REPORT

submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ELECTRICAL AND ELECTRONICS ENGINEERING

by

AARON CHACKO ABRAHAM (20BEE1035)

PARITOSH TAOSE (20BEE1082)

SIDHARTH MISHRA (20BEE1091)

ARITRA BISWAS (20BEE1092)

under the guidance of

Dr. Jayapragash R



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF ELECTRICAL ENGINEERING

VIT CHENNAI

November 2023

ACKNOWLEDGEMENT

It gives us an immense pleasure to bring out the project report on **Solar Battery Charger using Buck Converter and MPPT Algorithm**. We greatly accept this opportunity to express our deep sense of gratitude to our guide **Prof. Jayapragash R., Head of Department of the School of Electrical Engineering (SELECT)** for his kind co-operation and meticulous care with which he examined our work throughout the duration of our project.

I am extremely grateful to **Dr. Senthil Kumar N., Dean of the School of Electrical Engineering (SELECT), VIT University Chennai**, for extending the facilities of the schools towards my internship and for their unstinting support.

I also take this opportunity to thank all the faculty of the school for their support and their wisdom imparted to us throughout the courses till date.

I thank my parents, family, and friends for bearing with me throughout the course of my internship and for the opportunity they provided us in undergoing this course in such a prestigious institution.

ABSTRACT

The design and implementation of a Maximum Power Point Tracking (MPPT) Solar Charge Controller using Arduino is a significant leap in solar power charging technology. Traditional solar panels often produce voltage levels higher than necessary for charging 12V batteries, resulting in wasted energy. The MPPT Solar Charge Controller addresses this inefficiency by harnessing the Maximum Power Point Tracking technology, converting excess voltage into current to significantly enhance charging efficiency.

This project delves into the intricacies of solar power charging, elucidating the nuances of MPPT technology. Leveraging Arduino and a combination of active-passive electronic components, the schematic and printed circuit board (PCB) for the MPPT Charge Controller are meticulously designed. The Arduino UNO is programmed using C code to visualize crucial charging parameters on a 20x4 LCD screen, offering a comprehensive overview of the system's performance.

The code encompasses functionalities for measuring Solar Panel Voltage, Current, Power, Battery Voltage, Charger state, State of Charge (SOC), Pulse Width Modulation (PWM) duty cycle, and load status. The integration of these parameters provides a holistic understanding of the MPPT Solar Charge Controller's operation. The project is specifically tailored for a 50W solar panel, optimized for charging commonly used 12V lead-acid batteries.

Throughout the development, testing is employed to ensure the controller's efficacy in diverse conditions. The comprehensive nature of the design, from conceptualization to programming, positions this MPPT Solar Charge Controller as a versatile solution for harnessing solar energy efficiently. This project not only contributes to sustainable energy practices but also serves as an educational resource for enthusiasts seeking to understand and implement MPPT technology in solar charging systems. The results of the day-long testing further validate the effectiveness of the designed controller, showcasing its suitability for practical applications in renewable energy setups. As the world continues to embrace sustainable technologies, this project offers a practical and accessible avenue for individuals and communities to optimize solar power harvesting, exemplifying the power of innovation in advancing clean energy solutions.

TABLE OF CONTENTS

S.NO	CONTENT	PAGE NO.
1	List of Figures	iv
2	Abbreviations and Nomenclature	v
3	CHAPTER I 1. Introduction to the Project 1.1 Introduction 1.2 Motivation 1.3 Objectives 1.4 Scope of Work	1 1 2 2 2 2
4	CHAPTER II 2. Description of Project 2.1 Overview 2.2 Requirements	3 3 3 3
5	CHAPTER III 3. Design of the Project 3.1 Block Diagram 3.2 Circuit Diagram 3.3 Arduino Code 3.4 Working of the Project	5 5 5 5 10
6	CHAPTER IV 4. Results Obtained in the Project 4.1 Analytical Calculation 4.2 Hardware and Simulation Results	12 12 12 13
7	CHAPTER V 5. Conclusion 5.1 Conclusion 5.2 Future Enhancements 5.3 Recommendations 5.4 References	14 14 14 14 14

LIST OF FIGURES

- 2.1 IRFZ44ZPBF MOSFET and its respective data sheet
- 2.2 IR2104S MOSFET Gate Driver IC and its respective data sheet
- 2.3 Arduino Uno
- 3.1 Circuit Diagram of the Solar Charger with the buck converter
- 3.2 Fabricated PCB
- 4.1 Output in the serial monitor of ARDUINO IDE
- 4.2 Output of 11.99V seen on voltmeter as required
- 4.3 Solar Panel kept on the window for charging

ABBREVIATIONS AND NOMENCLATURE

1. PWM: Pulse Width Modulation Controller
2. MPPT: Maximum Power Point Tracking Controller
3. LCD: Liquid Crystal Display
4. SoC: State of Charge
5. MOSFET: Metal Oxide Semiconductor Field Effect Transistor
6. IC: Integrated Circuit
7. LED: Light Emitting Diode

CHAPTER I

Introduction to Project

1.1 INTRODUCTION

In recent years, the global trajectory towards renewable energy has catalysed the evolution and application of solar power systems, with a pronounced emphasis on the indispensable role of solar charge controllers in ensuring the effectiveness and durability of solar panel systems equipped with energy storage capabilities. These controllers serve as pivotal intermediaries in the intricate interplay between solar panels and batteries, meticulously regulating the flow of electrical current to shield against potential damage from overcharging or undercharging. The safeguarding function of these controllers is paramount in extending the overall lifespan of batteries, which constitute integral components of solar energy storage, while simultaneously ensuring an uninterrupted power supply by preventing the reverse flow of energy from the battery through the solar panel during the night. The landscape of solar charge controllers encompasses three distinct types: the Simple On-Off Controller (ON OFF), the Pulse Width Modulation Controller (PWM), and the Maximum Power Point Tracking Controller (MPPT). Among these, the MPPT controller stands out as a sophisticated solution, revolutionizing the charging process by dynamically adjusting the voltage and current of the solar panel to align with the optimal charging voltage of the battery. In a departure from older controller types, the MPPT controller showcases its intelligence by converting excess voltage into additional current, a process that would otherwise be wasted in traditional Pulse Width Modulation controllers. One of the standout features of most solar panels is their ability to generate higher voltages than necessary for charging batteries; for example, a 12V charging panel can produce 16 to 18 volts, although only about 14.6 volts are required for charging a typical 12V battery. The MPPT controller ingeniously capitalizes on this surplus voltage by transforming it into additional current, thereby not only expediting the charging process but also amplifying the overall power output of the solar system. Furthermore, the adaptability of the MPPT controller to fluctuating environmental conditions sets it apart from conventional charge controllers. It can dynamically respond to changes in sunlight intensity, temperature, and other factors, ensuring optimal performance across diverse operating conditions. As the world embarks on an inexorable transition towards sustainable energy solutions, the meticulous design and implementation of MPPT solar charge controllers become increasingly pertinent. This project represents a deep dive into the intricate details of designing an efficient MPPT controller, with the overarching aim of contributing to the progression of solar power technology and the broader adoption of clean and renewable energy sources. The essence of these controllers lies in their capacity to maximize power extraction from solar panels, augment charging efficiency, and seamlessly adapt to variable environmental conditions, thereby playing a pivotal role in the global paradigm shift towards cleaner and sustainable energy alternatives.

1.2 MOTIVATION

The impetus driving the development of a solar charger with a buck converter and MPPT algorithm is deeply rooted in the global imperative to transition towards sustainable energy sources. The escalating environmental concerns and the finite nature of traditional energy resources underscore the critical need for eco-friendly alternatives. By embarking on this project, we aim to contribute to the paradigm shift towards clean energy adoption. The motivation lies in mitigating the environmental impact associated with conventional energy consumption, fostering a greater reliance on renewable sources. The intersection of technology and sustainability fuels our commitment to developing a solar charger that not only harnesses the power of the sun efficiently but also aligns with the ethos of responsible energy usage.

1.3 OBJECTIVES

The multifaceted objectives of this solar charger project are intricately designed to address key challenges in solar energy utilization. Firstly, the project aims to engineer a highly efficient charging system capable of seamlessly interfacing with diverse battery technologies. The incorporation of a Buck converter ensures precise voltage regulation, enhancing compatibility with a spectrum of battery types and configurations. Simultaneously, the integration of a sophisticated Maximum Power Point Tracking (MPPT) algorithm is envisioned to optimize the solar panel's performance, dynamically adjusting for varying environmental conditions to extract the maximum power available. Furthermore, the utilization of Arduino technology serves to infuse intelligence into the system, facilitating real-time monitoring, adaptive control, and the potential for remote management. The overarching objective is to deliver a solar charging solution that is not only technically adept but also adaptable and scalable, catering to a broad spectrum of applications from small-scale electronics to off-grid power systems.

1.4 SCOPE OF WORK

The expansive scope of this solar charger project spans the entire spectrum of design, development, and validation, with a comprehensive focus on functionality, adaptability, and user-friendliness. The initial phase entails meticulous solar panel selection, considering factors such as efficiency, size, and compatibility. The design intricacies of the Buck converter circuit, a critical component for effective voltage regulation, demand careful consideration and optimization. Simultaneously, the implementation and fine-tuning of the MPPT algorithm require a deep understanding of solar panel characteristics and environmental factors. The integration of Arduino into the system adds a layer of sophistication, necessitating programming for intelligent battery management, charge control, and real-time data acquisition. The testing phase involves rigorous simulations and practical experiments, evaluating the system's performance under diverse conditions to ensure robustness and reliability. The final deliverables encompass comprehensive documentation, providing clear instructions for assembly, operation, and troubleshooting. This ensures that the project is not just a technical achievement but a practical solution that can be replicated and utilized by the broader community of DIY enthusiasts, students, and renewable energy advocates.

CHAPTER II

Description of Project

2.1 OVERVIEW

The project aims to construct a Maximum Power Point Tracking (MPPT) Solar Charge Controller using Arduino and a combination of active-passive electronics. The MPPT technology becomes crucial as most solar panels generate a significantly higher voltage than necessary for charging 12V batteries. Typically, a 12V charging panel produces 16 to 18 volts, while only around 14.6 volts are required for charging. This excess voltage often goes unused, leading to inefficiencies. By implementing MPPT Charging Technology, the project seeks to convert surplus voltage into additional current, thereby increasing overall charging efficiency.

The project unfolds in several phases. Firstly, it delves into the intricacies of Solar Power Charging Technology and comprehensively explores MPPT Charging Technology. Subsequently, the focus shifts to designing the schematic and printed circuit board (PCB) for the MPPT Charge Controller, employing Arduino and various electronic components. The Arduino Nano is programmed using C code to visualize essential charging parameters on a 20×4 LCD Screen. The code encompasses functions for measuring Solar Panel Voltage, Current, Power, Battery Voltage, Charger state, State of Charge (SOC), Pulse Width Modulation (PWM) duty cycle, and load status.

Following the design and programming phases, the project moves on to practical testing. The MPPT Solar Charge Controller is subjected to real-world conditions throughout the day to assess its functionality and efficiency. The design is specifically tailored for a 50W solar panel, intending to charge a commonly used 12V lead-acid battery. The comprehensive approach of this project, from theoretical understanding to practical implementation and testing, aims to contribute to the advancement of solar power technology and facilitate the adoption of clean and renewable energy sources.

2.2 REQUIREMENTS

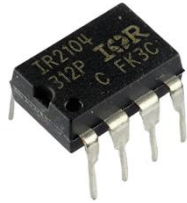
1. **Solar Panel:** Acts as the energy source.
2. **Synchronous Buck Converter:** Utilizes IRFZ44ZPBF MOSFET for voltage conversion.



Absolute Maximum Ratings			
Parameter	Max.	Units	
I_D @ $T_C = 25^\circ\text{C}$	51	A	Continuous Drain Current, $V_{GS} @ 10\text{V}$ (Silicon Limited)
I_D @ $T_C = 100^\circ\text{C}$	36	A	Continuous Drain Current, $V_{GS} @ 10\text{V}$ (See Fig. 9)
I_{AS}	200	A	Pulsed Drain Current $\text{\textcircled{D}}$
P_D @ $T_C = 25^\circ\text{C}$	80	W	Maximum Power Dissipation
	0.53	$^\circ\text{C/W}$	Linear Derating Factor
V_{GS}	± 20	V	Gate-to-Source Voltage
E_{AS}	86	mJ	Single Pulse Avalanche Energy (Thermally Limited) $\text{\textcircled{D}}$
$E_{AS}(\text{tested})$	105	mJ	Single Pulse Avalanche Energy Tested Value $\text{\textcircled{D}}$
I_{AS}	See Fig. 12a, 12b, 15, 16	A	Avalanche Current $\text{\textcircled{D}}$
E_{RHS}		mJ	Repetitive Avalanche Energy $\text{\textcircled{D}}$
T_J	-55 to $+175$	$^\circ\text{C}$	Operating Junction and Storage Temperature Range
t_{SOG}	300 (1.6mm from case.)		Soldering Temperature, for 10 seconds
	10 lbf-in (1.1N-m)		Mounting torque, 6-32 or M3 screw
Thermal Resistance			
Parameter	Typ.	Max.	Units
R_{JC}	—	1.87	$^\circ\text{C/W}$
R_{CS}	0.50	—	Case-to-Sink, Flat, Greased Surface
R_{JA}	—	62	Junction-to-Ambient
R_{JA}	—	40	Junction-to-Ambient (PCB Mount, steady state) $\text{\textcircled{D}}$

2.1 IRFZ44ZPBF MOSFET and its respective data sheet

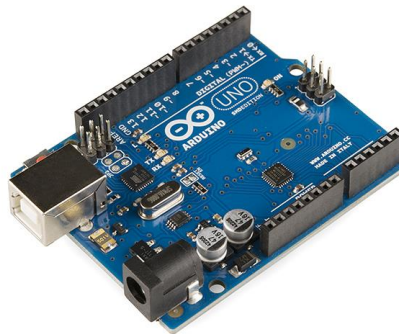
3. **IR2104S MOSFET Gate Driver IC:** Drives the high-side and low-side MOSFETs in a half-bridge configuration.



IR2104(S)&(PbF)				
International Rectifier				
Absolute Maximum Ratings				
Absolute maximum ratings indicate sustained limits beyond which damage to the device may occur. All voltage parameters are absolute voltages referenced to COM. The thermal resistance and power dissipation ratings are measured under board mounted and still air conditions.				
Symbol	Definition	Min.	Max.	Units
V_B	High side floating absolute voltage	-0.3	625	V
V_S	High side floating supply offset voltage	$V_B - 25$	$V_B + 0.3$	
V_{HO}	High side floating output voltage	$V_S - 0.3$	$V_B + 0.3$	
V_{CC}	Low side and logic fixed supply voltage	-0.3	25	
V_{LO}	Low side output voltage	-0.3	$V_{CC} + 0.3$	
V_{IN}	Logic input voltage (IN & SD)	-0.3	$V_{CC} + 0.3$	
dV_S/dt	Allowable offset supply voltage transient	—	50	V/ns
P_D	Package power dissipation @ $T_A \leq +25^\circ\text{C}$	—	1.0	W
	(8 lead PDIP)	—	0.625	
	(8 lead SOIC)	—	125	$^\circ\text{C/W}$
R_{thJA}	Thermal resistance, junction to ambient	—	200	
	(8 lead PDIP)	—	150	$^\circ\text{C}$
T_J	Junction temperature	—	150	
T_S	Storage temperature	-55	150	
T_L	Lead temperature (soldering, 10 seconds)	—	300	

2.2 IR2104S MOSFET Gate Driver IC and its respective data

4. **Arduino Uno:** Controls the MPPT algorithm and generates gate pulses for the MOSFETs.



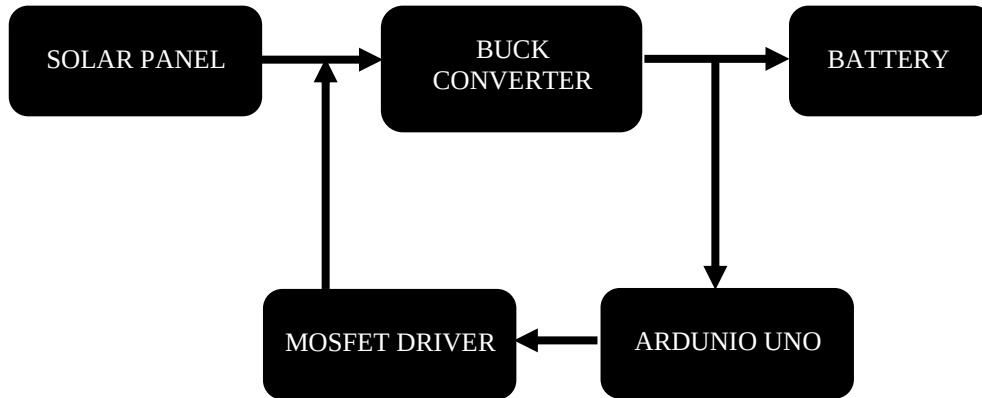
2.3 Arduino Uno

5. **Voltage Divider Circuit:** Measures solar panel and battery voltages.
6. **Synchronous Buck Converter Working Principle:** The synchronous buck converter steps down the input voltage from the solar panel to a lower output voltage for charging the battery. It employs synchronous rectification, using high-side and low-side MOSFETs along with an inductor and capacitor for voltage conversion. The IR2104S gate driver IC efficiently controls these MOSFETs, ensuring precise switching and preventing shoot-through currents.
7. **MPPT Algorithm:** The Arduino Uno reads the solar panel and battery voltages using a voltage divider circuit. The MPPT algorithm implemented in the Arduino adjusts the duty cycle of the PWM signal sent to the IR2104S, optimizing the power harvested from the solar panel. The algorithm ensures that the solar panel operates at its maximum power point, enhancing the overall efficiency of the system.

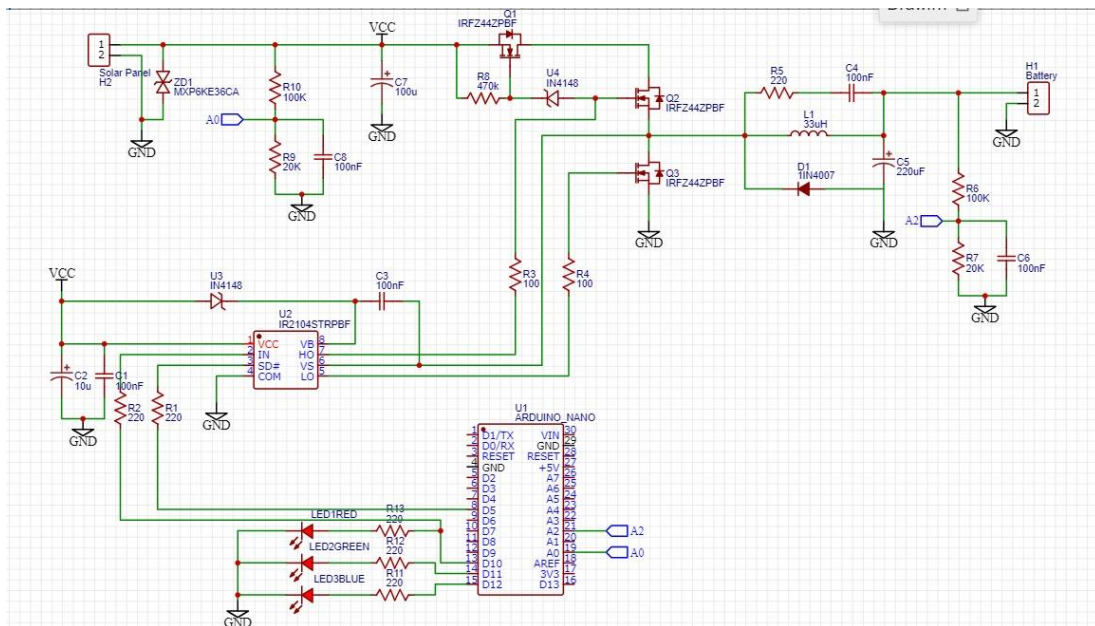
CHAPTER III

Design of Project

3.1 BLOCK DIAGRAM



3.2 CIRCUIT DIAGRAM



3.1 Circuit Diagram of the Solar Charger with the buck converter

3.3 ARDUINO CODE

```
#include "TimerOne.h"
#include <LiquidCrystal_I2C.h>
#include <Wire.h>
#define ENABLE_DATALOGGER 0
#define LOAD_ALGORITHM 1

#define SOL_AMPS_CHAN 1
#define SOL_VOLTS_CHAN 0
#define BAT_VOLTS_CHAN 2
#define AVG_NUM 8
// #define
SOL_AMPS_SCALE 0.026393581
```

```

#define SOL_VOLTS_SCALE
0.029296875
#define BAT_VOLTS_SCALE
0.029296875
#define PWM_PIN 6
#define PWM_ENABLE_PIN 5
#define PWM_FULL 1023
#define PWM_MAX 100
#define PWM_MIN 60
#define PWM_START 90
#define PWM_INC 1

#define TRUE 1
#define FALSE 0
#define ON TRUE
#define OFF FALSE

#define TURN_ON_MOSFETS
digitalWrite(PWM_ENABLE_PIN,
HIGH)
#define TURN_OFF_MOSFETS
digitalWrite(PWM_ENABLE_PIN, LOW)
#define ONE_SECOND 50000
#define LOW_SOL_WATTS 5.00
#define MIN_SOL_WATTS 0.10
#define MIN_BAT_VOLTS 11.00
#define MAX_BAT_VOLTS 14.10
#define BATT_FLOAT 13.60
#define HIGH_BAT_VOLTS 13.00
#define LVD 11.5
#define OFF_NUM 9
#define LED_YELLOW 12
#define LED_GREEN 11
#define LED_RED 10
#define LOAD_PIN 4
#define BACK_LIGHT_PIN 3
float AcsValue, ss = 0.0;
#define PWM_ICON 7
float sol_amps;
float sol_volts;
float bat_volts;
float sol_watts;
float old_sol_watts = 0;
unsigned int seconds = 0;
unsigned int prev_seconds = 0;
unsigned int interrupt_counter =
0;
unsigned long time = 0;
int delta = PWM_INC;
int pwm = 0;
int back_light_pin_State = 0;
boolean load_status = false;

enum charger_mode {off, on, bulk,
bat_float} charger_state;

LiquidCrystal_I2C lcd(0x27, 20,
4);
void setup()
{
    pinMode(PWM_ENABLE_PIN,
OUTPUT);
    TURN_OFF_MOSFETS;
    charger_state = off;

    pinMode(LED_RED, OUTPUT);
    pinMode(LED_GREEN, OUTPUT);
    pinMode(LED_YELLOW, OUTPUT);
    Timer1.initialize(20);
    Timer1.pwm(PWM_PIN, 0);
    Timer1.attachInterrupt(callback
);
    Serial.begin(9600);
    pwm = PWM_START;
    pinMode(BACK_LIGHT_PIN, INPUT);
    pinMode(LOAD_PIN, OUTPUT);
    digitalWrite(LOAD_PIN, LOW);
    digitalWrite(BACK_LIGHT_PIN,
LOW);
}
void loop()
{
    read_data();
    run_charger();
    print_data();
    load_control();
    led_output();
}

int read_adc(int channel)
{
    int sum = 0;
    int temp;
    int i;
    for (i = 0; i < AVG_NUM; i++)
    {
        temp = analogRead(channel);
        sum += temp;
        delayMicroseconds(50);
    }
    return (sum / AVG_NUM);
}

float read_adccc()
{
    int i;
    float AcsValue, ss = 0.0;
    for (i = 0; i < AVG_NUM; i++)
    {
        AcsValue = analogRead(A1);
        ss += mapf(AcsValue, 510,
580, 0.0, 5.0);
    }
}

```

```

        delayMicroseconds(50);
    }
    Serial.println(AcsValue);
    Serial.println(ss / AVG_NUM);
    return (ss / AVG_NUM);
}

void read_data(void)
{
    //sol_amps = -read_adccc();
    sol_amps = 0;
    sol_volts =
    read_adc(BAT_VOLTS_CHAN) *
    SOL_VOLTS_SCALE;
    bat_volts =
    read_adc(SOL_VOLTS_CHAN) *
    BAT_VOLTS_SCALE;
    sol_watts = sol_amps *
    sol_volts ;
}

void callback()
{
    if (interrupt_counter++ >
    ONE_SECOND)
    {
        interrupt_counter = 0;
        seconds++;
    } }

void set_pwm_duty(void)
{
    if (pwm > PWM_MAX) {
        pwm = PWM_MAX;
    }
    else if (pwm < PWM_MIN) {
        pwm = PWM_MIN;
    }
    if (pwm < PWM_MAX) {
        Timer1.pwm(PWM_PIN,
        (PWM_FULL * (long)pwm / 100),
        20);
    }
    else if (pwm == PWM_MAX) {
        Timer1.pwm(PWM_PIN,
        (PWM_FULL - 1), 20);
    }
}

void run_charger(void)
{
    static int off_count = OFF_NUM;
    switch (charger_state)
    {
        case on:

```

```

        if (sol_watts <
        MIN_SOL_WATTS)
        {
            charger_state = off;
            off_count = OFF_NUM;
            TURN_OFF_MOSFETS;
        }
        else if (bat_volts >
        (BATT_FLOAT - 0.1))
        {
            charger_state = bat_float;
        }
        else if (sol_watts <
        LOW_SOL_WATTS) {
            pwm = PWM_MAX;
            set_pwm_duty();
        }
        else {
            pwm = ((bat_volts * 10) /
            (sol_volts / 10)) + 5;
            charger_state = bulk;
        }
        break;
        case bulk:
            if (sol_watts <
            MIN_SOL_WATTS)
            {
                charger_state = off;
                off_count = OFF_NUM;
                TURN_OFF_MOSFETS;
            }
            else if (bat_volts >
            BATT_FLOAT)
            {
                charger_state = bat_float;
            }
            else if (sol_watts <
            LOW_SOL_WATTS)
            {
                charger_state = on;
                TURN_ON_MOSFETS;
            }
            else {
                if (old_sol_watts >=
                sol_watts)
                {
                    delta = -delta;
                }
                pwm += delta;
                old_sol_watts = sol_watts;
                set_pwm_duty();
            }
            break;
            case bat_float:
                if (sol_watts <
                MIN_SOL_WATTS)

```

```

        {
            charger_state = off;
            off_count = OFF_NUM;
            TURN_OFF_MOSFETS;
            set_pwm_duty();
        }
        else if (bat_volts >
BATT_FLOAT)
        {
            TURN_OFF_MOSFETS;
            pwm = PWM_MAX;
            set_pwm_duty();
        }
        else if (bat_volts <
BATT_FLOAT)
        {
            pwm = PWM_MAX;
            set_pwm_duty();
            TURN_ON_MOSFETS;
            if (bat_volts < (BATT_FLOAT
- 0.1))
            {
                charger_state = bulk;
            }
            break;
            case off:
                TURN_OFF_MOSFETS;
                if (off_count > 0)
                {
                    off_count--;
                }
            else if ((bat_volts >
BATT_FLOAT) && (sol_volts >
bat_volts)) {
                charger_state = bat_float;
                TURN_ON_MOSFETS;
            }
            else if ((bat_volts >
MIN_BAT_VOLTS) && (bat_volts <
BATT_FLOAT) && (sol_volts >
bat_volts)) {
                charger_state = bulk;
                TURN_ON_MOSFETS;
            }
            break;
            default:
                TURN_OFF_MOSFETS;
            break;
        }
    }
    void load_control()
    {
        #if LOAD_ALGORITHM == 0
            load_on(sol_watts <
MIN_SOL_WATTS && bat_volts >
LVD);
        #else
            load_on(sol_watts >
MIN_SOL_WATTS && bat_volts >
BATT_FLOAT);
        #endif
    }
    void load_on(boolean new_status)
    {
        if (load_status != new_status)
        {
            load_status = new_status;
            digitalWrite(LOAD_PIN,
new_status ? HIGH : LOW);
        }
    }
    void print_data(void)
    {
        Serial.print(seconds, DEC);
        Serial.print(" ");
        Serial.print("Charging = ");
        if (charger_state == on)
            Serial.print("on ");
        else if (charger_state == off)
            Serial.print("off ");
        //else if (charger_state ==
bulk) Serial.print("bulk ");
        else if (charger_state == bulk)
            Serial.print("on ");
        else if (charger_state ==
bat_float) Serial.print("float");
        Serial.print(" ");
        //Serial.print("pwm = ");
        if (charger_state == off){}
        //Serial.print(0, DEC);
        //else
        //Serial.print(pwm, DEC);
        //Serial.print(" ");

        Serial.print("Current (panel) =
");
        Serial.print(sol_amps);
        Serial.print(" ");
        Serial.print("Voltage (panel)
= ");
        Serial.print(sol_volts);
        Serial.print(" ");
        Serial.print("Power (panel) =
");
        Serial.print(sol_watts);
        Serial.print(" ");
        Serial.print("Battery Voltage
= ");
        Serial.print(bat_volts);
        Serial.print(" ");
        Serial.print("\n\r");
    }
}

```



```

void light_led(char pin)
{
    static char last_lit;
    if (last_lit == pin)
        return;
    if (last_lit != 0)
        digitalWrite(last_lit,
LOW);
    digitalWrite(pin, HIGH);
    last_lit = pin;
}
void led_output(void)
{
    static char last_lit;
    if (bat_volts > 14.1 )
        light_led(LED_YELLOW);
    else if (bat_volts > 11.9)
        light_led(LED_GREEN);
    else
        light_led(LED_RED);
}
void lcd_display()
{
    static bool
current_backlight_state = -1;
    back_light_pin_State =
digitalRead(BACK_LIGHT_PIN);
    if (current_backlight_state !=
back_light_pin_State) {
        current_backlight_state =
back_light_pin_State;
        if (back_light_pin_State ==
HIGH)
            lcd.backlight();
        else
            lcd.noBacklight();
    }
    if (back_light_pin_State ==
HIGH)
    {
        time = millis();
    }
    lcd.setCursor(0, 1);
    lcd.print(sol_volts);
    lcd.print("V ");
    lcd.setCursor(0, 2);
    lcd.print(sol_amps);
    lcd.print("A");
    lcd.setCursor(0, 3);
    lcd.print(sol_watts);
    lcd.print("W ");
    lcd.setCursor(8, 1);
    lcd.print(bat_volts);
    lcd.setCursor(8, 2);
    if (charger_state == on)
        lcd.print("on ");
        else if (charger_state == off)
            lcd.print("off ");
        else if (charger_state == bulk)
            lcd.print("bulk ");
        else if (charger_state ==
bat_float)
        {
            lcd.print(" ");
            lcd.setCursor(8, 2);
            lcd.print("float");
        }
        int pct = 100.0 * (bat_volts -
11.3) / (12.7 - 11.3);
        if (pct < 0)
            pct = 0;
        else if (pct > 100)
            pct = 100;

        lcd.setCursor(12, 0);
        lcd.print((char)(pct * 5 /
100));
        lcd.setCursor(8, 3);
        pct = pct - (pct % 10);
        lcd.print(pct);
        lcd.print("% ");
        lcd.setCursor(15, 0);
        lcd.print("PWM");
        lcd.setCursor(19, 0);
        lcd.write(PWM_ICON);
        lcd.setCursor(15, 1);
        lcd.print(" ");
        lcd.setCursor(15, 1);
        if ( charger_state == off)
            lcd.print(0);
        else
            lcd.print(pwm);
        lcd.print("% ");
        lcd.setCursor(15, 2);
        lcd.print("Load");
        lcd.setCursor(15, 3);
        if (load_status)
        {
            lcd.print("On ");
        }
        else
        {
            lcd.print("Off ");
        }
        spinner();
        backLight_timer();
    }
}

void backLight_timer()
{
    if ((millis() - time) <= 15000)
        lcd.backlight();
}

```

```

else
    lcd.noBacklight();
}
void spinner(void)
{
    static int cspinner;
    static char spinner_chars[] = {
        '*', '*', '*', ' ', ' '};
    cspinner++;
}

```

```

float mapf(float x, float
in_min, float in_max, float
out_min, float out_max) {
    return (x - in_min) * (out_max
- out_min) / (in_max - in_min) +
out_min;
}

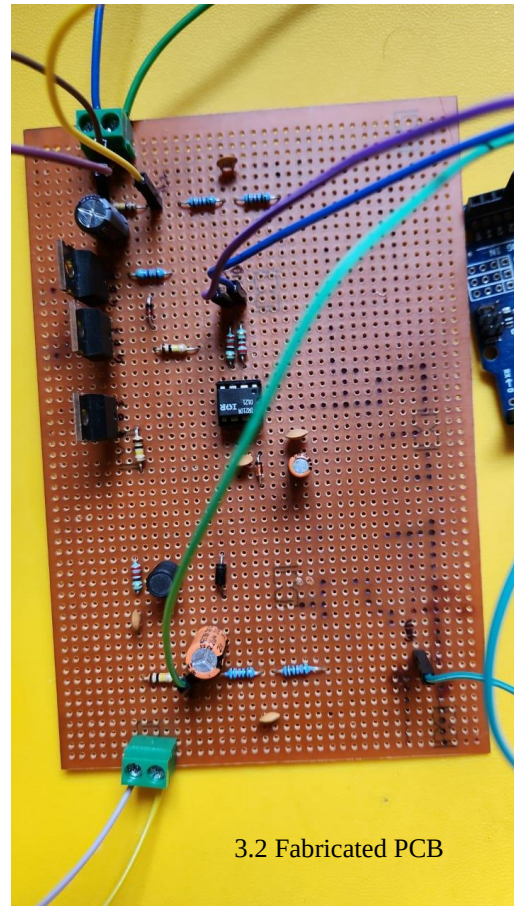
+;
lcd.print(spinner_chars[cspinne
r % sizeof(spinner_chars) - 1]);

```

3.4 WORKING OF THE PROJECT

The voltage originating from the solar panel serves as the input voltage for the system. The buck converter incorporates synchronous MOSFET switches Q4 and Q5, along with the energy storage components coil L1 and capacitors C4 and C9. L1, in conjunction with C4, functions to smooth the switching current and regulate the output voltage. Additionally, damping networks comprising capacitor C3 and resistor R4 are employed to mitigate coil voltage ringing induced by the inductor's switching current. MOSFET Q3 is strategically integrated to prevent reverse battery current flow to the solar panels during nighttime.

The activation of Q3 is contingent upon the activation of Q4 through voltage D2, and the deactivation of Q3 coincides with the deactivation of Q4, facilitated by the drain of gate voltage via resistor R3. The IC IR2104, serving as a half-bridge MOSFET gate driver, is instrumental in propelling the high and low side MOSFETs through the PWM signal from Arduino Pin D6. Furthermore, the IR2104 can be deactivated through a control signal from Arduino Pin D5 on pin 3. The bootstrap circuit, comprising D4 and C6, is integral to providing the high side gate control voltage for Q3 and Q4. Software control of the PWM duty cycle is implemented to ensure that it never reaches 100% or remains constantly on, limiting it to 99.9% to sustain the charge pump operation. Two voltage divider circuits, denoted by R1, R2, and R7, R8, are employed to gauge the voltage levels of the solar panel and the battery.



The output signals from these dividers are fed to the analog pins A0 and A2 of the Arduino. Diode D3 is incorporated to enhance converter efficiency, while D1 and D5 serve as TVS diodes for overvoltage protection on the solar panel and load side, respectively. MOSFET Q2 governs load control, with its driver circuit comprising transistor Q1, along with resistors R5 and R6. To monitor the current from the solar panel, an ACS712 current sensor is utilized, transmitting the data to analog pin A1 of the Arduino. The system's charging status is visually conveyed through three LEDs connected to the digital pins of the microcontroller. Additionally, an LCD backlight switch is integrated, enabling the user to activate the backlight for a 15-second duration upon pressing the switch, after which it automatically turns off.

CHAPTER IV

Results obtained in the project

4.1 ANALYTICAL CALCULATION

$$\text{Parasitic Capacitance } C_o = \frac{C_1}{m^2 - 1}$$

$$\text{Parasitic Inductance } L_o = \frac{m^2 - 1}{(2\pi f_o)^2 C_1}$$

$$[C_{snubber} = 3C_o = \text{Starting snubber capacitor value}]$$

$$(V_s - V_o)D + (-V_o)(1 - D) = 0$$

$$V_L = L \frac{dI_L}{dt} \quad | \quad V_o = L \frac{\Delta I_L}{(1-D)T_s}$$

$$L = \frac{V_o(1-D)T_s}{\Delta I_L} \Rightarrow L = \frac{V_o(1-D)}{\Delta I_L f_s}$$

$$\Delta I_L = \frac{12 \cdot 0.5}{50 \cdot 10^3 \cdot 33 \cdot 10^{-6}} \Rightarrow \Delta I_L = 3.64 \text{ A}$$

12V Lead Acid Battery has a discharge rate of 20hr. Voltage rated at 12V, 7Ah.

$$\Delta I_L = \frac{3.64}{7} * 100 = 52\% \text{ of } I_o. \text{ After using the snubber circuit, } \Delta I_L \text{ is reduced.}$$

$$Q = C\Delta V_C$$

$$\Rightarrow \frac{1}{2} * \frac{1}{2} (DT_s) * \frac{\Delta I_L}{2} = C\Delta V_C \quad | \quad \Delta V_C = \frac{0.021}{12} * 100, \Delta V_C = 0.2\% \text{ of } V_o$$

$$\Rightarrow \frac{1}{8} * (DT_s \Delta I_L) = C\Delta V_C = \frac{1}{8} * \frac{0.5 \cdot 3.64}{50 \cdot 10^3 \cdot 2 \cdot 10^{-6}} = \Delta V_C$$

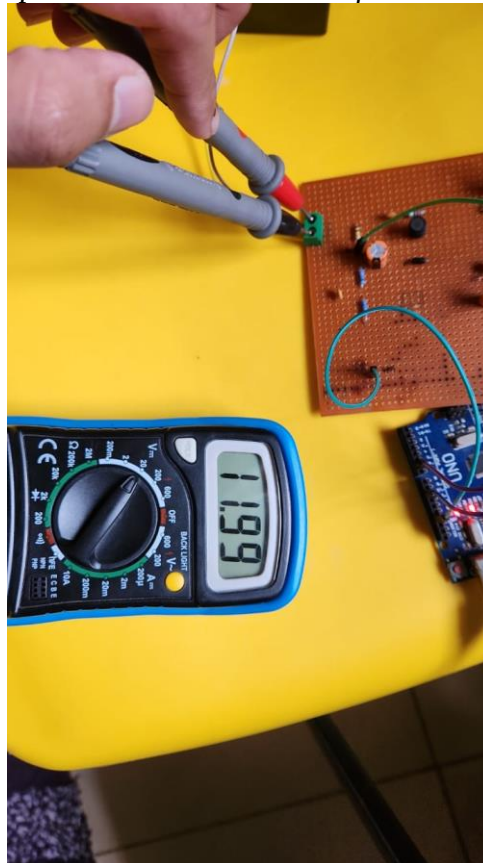
$$\Delta V_C = 0.227V$$

$$\text{Battery Rating} = P = V_o I_o = (12V) * (7Ah) = 84Wh$$

4.2 HARDWARE AND SIMULATION RESULTS

Output		Serial Monitor x				
Message (Enter to send message to 'Arduino Uno' on 'COM8')						
					Both NL & CR	9600 baud
128	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.90	Power (panel) = 44.22	Battery Voltage = 12.01	
128	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.93	Power (panel) = 44.29	Battery Voltage = 12.01	
128	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.93	Power (panel) = 44.29	Battery Voltage = 12.01	
128	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.90	Power (panel) = 44.22	Battery Voltage = 12.01	
129	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.93	Power (panel) = 44.29	Battery Voltage = 12.01	
129	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.90	Power (panel) = 44.22	Battery Voltage = 11.98	
129	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.93	Power (panel) = 44.29	Battery Voltage = 12.01	
129	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.90	Power (panel) = 44.22	Battery Voltage = 12.01	
129	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.93	Power (panel) = 44.29	Battery Voltage = 12.01	
129	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.90	Power (panel) = 44.22	Battery Voltage = 12.01	
129	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.93	Power (panel) = 44.29	Battery Voltage = 12.01	
130	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.90	Power (panel) = 44.22	Battery Voltage = 12.01	
130	Charging = on	Current (panel) = 2.34	Voltage (panel) = 18.90	Power (panel) = 44.22	Battery Voltage = 12.01	

4.1 Output in the serial monitor of ARDUINO IDE



4.2 Output of 11.99V seen on voltmeter as required



4.3 Solar Panel kept on the window for charging

CHAPTER V

Conclusion

5.1 CONCLUSION

The project successfully demonstrates the integration of an MPPT algorithm with a synchronous buck converter for efficient solar battery charging. The Arduino Uno, acting as the control unit, dynamically adjusts the charging parameters based on solar conditions, optimizing power transfer and ensuring effective battery charging.

5.2 FUTURE ENHANCEMENTS

Future improvements may include real-time data logging, integration of additional sensors for environmental conditions, and the implementation of communication protocols for remote monitoring.

5.3 RECOMMENDATIONS

1. Ensure proper safety precautions when working with high voltages and currents.
2. Refer to datasheets and application notes for precise component connections and operational characteristics.
3. Periodically monitor and analyze the system's performance for continuous optimization.

5.4 REFERENCES

1. S. D. Patri, S. H. Vali and V. Rafi, "Battery charging using a hill climbing MPPT algorithm and heuristic algorithm (MPPT)," 2021 International Conference on Recent Trends on Electronics, Information, Communication & Technology (RTEICT), Bangalore, India, 2021, pp. 91-96, doi: 10.1109/RTEICT52294.2021.9573849.
2. A. Badis, M. N. Mansouri and M. H. Boujmil, "A genetic algorithm optimized MPPT controller for a PV system with DC-DC boost converter," 2017 International Conference on Engineering & MIS (ICEMIS), Monastir, Tunisia, 2017, pp. 1-6, doi: 10.1109/ICEMIS.2017.8273010.
3. Patil S.N. and R. C. Prasad, "Design and development of MPPT algorithm for high efficient DC-DC converter for solar energy system connected to grid," 2015 International Conference on Energy Systems and Applications, Pune, India, 2015, pp. 228-233, doi: 10.1109/ICESA.2015.7503345.
4. P. K. Atri, P. S. Modi and N. S. Gujar, "Design and Development of Solar Charge Controller by Implementing two different MPPT Algorithm," 2021 International Conference on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT), Bhilai, India, 2021, pp. 1-5, doi: 10.1109/ICAECT49130.2021.9392426.